# Using the NIST Statistical Test Suite

## What the suite is

The NIST Statistical Test Suite (STS) is a free ANSI C program that runs 15 statistical tests on a stream of bits to see whether it looks random. It is the standard first check for hardware random number generators (RNGs) and software pseudorandom number generators (PRNGs), and you compile it yourself from NIST's source code.

Here is how it decides:

- Every test assumes the null hypothesis: the sequence is random.
- Each test turns your bits into a **P-value**: the probability that a perfect random generator would produce a sequence that looks *less* random than yours. A P-value near 0 means your data looks very non-random.
- You choose a significance level **α** (default 0.01). If $P \geq \alpha$ the sequence passes that test; if $P < \alpha$ it fails.
- Because of that, a perfect generator still fails about 1% of sequences at $\alpha = 0.01$. That is why you always test many sequences and judge the whole group (Step 5).

NIST's own workflow has five stages, and this guide follows them:

1. Pick the generator to evaluate.
2. Generate several binary sequences of a fixed length *n* and save them to a file.
3. Run the test suite on that file.
4. Look at the P-values the suite writes out.
5. Decide pass or fail by looking at the pass rate and the spread of P-values across all sequences.

The 15 tests, numbered as they appear in the program's menu (you will type these numbers later):

| # | Test | What it looks for |
|---|---|---|
| 1 | Frequency (monobit) | Overall balance of 0s and 1s |
| 2 | Block Frequency | Balance of 1s inside each M-bit block |
| 3 | Cumulative Sums | A running +1/−1 total that drifts too far from zero |
| 4 | Runs | Too many or too few runs of identical bits |
| 5 | Longest Run of Ones | Longest streak of 1s per block versus what is expected |
| 6 | Rank | Linear dependence among fixed-size bit matrices (32×32) |
| 7 | Spectral (DFT) | Repeating, periodic patterns, found with a Fourier transform |

| # | Test | What it looks for |
|---|---|---|
| 8 | Non-overlapping Template | Too many occurrences of a specific non-repeating bit pattern |
| 9 | Overlapping Template | Too many or too few runs of 1s of a given length |
| 10 | Universal (Maurer's) | Whether the sequence can be compressed, a proxy for per-bit entropy |
| 11 | Approximate Entropy | Frequency of m-bit versus (m+1)-bit overlapping patterns |
| 12 | Random Excursions | Visits to each state of the random walk, per cycle |
| 13 | Random Excursions Variant | Total visits to each state of the random walk |
| 14 | Serial | Whether every overlapping m-bit pattern is equally likely |
| 15 | Linear Complexity | Structure like a linear feedback shift register (LFSR) |

One limit to keep in mind: passing these tests does not prove a generator is secure. NIST says statistical testing is a useful first step but cannot replace cryptanalysis.

## Step 1: Download, unpack, and build

You need a C compiler and `make`, and the build produces one program called `assess`. Linux and macOS are the easiest; on Windows, run everything inside WSL (Windows Subsystem for Linux) with Ubuntu.

**1. Download the suite.** The link is on NIST's Documentation and Software page.

**2. Unpack it and move into the folder.** Run every later command from inside this folder, because the program uses relative paths such as `data/` and `experiments/`.

```
unzip sts-2_1_2.zip
cd sts-2.1.2
```

**3. Know what you are looking at.** You should see these items:

| Item | What it is |
|---|---|
| `makefile` | Build instructions |
| `src/` | C source for each test, plus the main program `assess.c` |
| `include/` | Header files and the settings file `defs.h` |
| `obj/` | Compiled object files (filled in by `make`) |
| `templates/` | Bit patterns used by the Non-overlapping Template test |
| `data/` | Input files to test, including five sample files (`data.pi`, `data.e`, |

| Item | What it is |
|---|---|
| | `data.sqrt2`, `data.sqrt3`, `data.sha1`) |
| `experiments/` | Where results are written; `AlgorithmTesting/` is the folder for results from your own data files |

**4. Build.**

```
make
```

When it finishes, an executable named `assess` appears in the project folder. Confirm it runs:

```
./assess
```

With no argument it prints a short usage message. That is success. It is not an error.

## Step 2: Your first run on the sample data

Before testing anything of your own, run the bundled file `data/data.pi` (the first million binary digits of π) and compare your output to NIST's published numbers. If they match, your build works. NIST supplies the sample data and results for exactly this purpose.

**1. Start the program.** The number after `assess` is the sequence length *n*, in bits, for each sequence it tests.

```
./assess 1000000
```

**2. Answer the menu prompts.** The program is interactive. Type the value in the right-hand column and press Enter after each one:

| The program asks | You type | Why |
|---|---|---|
| Generator options | `0` | Option 0 means "read from an input file"; the other options are NIST's built-in test generators |
| Input file | `data/data.pi` | The path to your data, relative to the STS folder |
| Apply all tests? (1 = yes, 0 = pick a subset) | `1` | Runs all 15 tests |
| Parameter adjustments (if shown) | `0` | Keeps the defaults (more on these in Step 4) |
| How many bit streams? | `1` | NIST's sample results use one stream of 1,000,000 bits |
| Input file format | `0` | `0` = ASCII file of the characters 0 and 1; `1` = binary file. `data.pi` is ASCII |

The run takes a little while. The Linear Complexity test is the slowest of the 15.

**3. Look at the raw results.** Each test writes its own folder inside `experiments/AlgorithmTesting/`, and each folder holds a `results.txt` (the P-values) and a `stats.txt` (the intermediate numbers behind them).

## Step 3: Test your own generator's output

To test your own source, save its output as one long file of bits, then tell `assess` how long each sequence is and how many sequences to read from the file. The program splits the file into sequences for you.

**1. Run your Python generator script to get its output.**

**2. Make the data file.** Two formats are supported: an **ASCII** file made only of the characters `0` and `1`, or a **binary** file where each byte holds 8 bits. Put the file in the `data/` folder. I would suggest using the ASCII output and pasting your random bitstream to a .txt file.

**3. Run the suite on it.**

```
./assess 1000000
```

| The program asks | You type |
|---|---|
| Generator options | `0` |
| Input file | `data/random.txt` |
| Apply all tests? | `1` |
| Parameter adjustments (if shown) | `0` |
| How many bit streams? | `100` |
| Input file format | `1` (binary) or `0` (ASCII) |

When the run finishes, go to Step 5 to read the results.

## Step 4: Configuration reference

Most runs only need the defaults, but you can change what is tested at three levels: the menu choices at run time, the per-test parameters, and the compile-time settings.

### Menu choices at run time

**Generator option.** `0` reads your input file. Options 1 through 9 make the program generate its own bits from built-in PRNGs (linear congruential, quadratic congruential I and II, cubic congruential, XOR, modular exponentiation, Blum-Blum-Shub, Micali-Schnorr, and a SHA-1 based generator). They need no data file, so they are handy for practice.

**Test selection.** Typing `1` runs all 15 tests. Typing `0` shows a second screen where you enter one digit per test, 15 digits in menu order, with `1` to run a test and `0` to skip it. For example, `101100000000000` runs only Frequency (1), Cumulative Sums (3) and Runs (4). NIST's own example `000000001000000` runs only test 9, Overlapping Template.

**Number of bit streams and file format.** These are the last two prompts, as in Steps 2 and 3.

### Per-test parameters

Six tests take a block or template length. The defaults below are the values NIST used for its published sample results.

| Test | Parameter | Default | NIST guidance |
|---|---|---|---|
| Block Frequency | Block length M | 128 | $M \geq 20$, $M > 0.01 \cdot n$, and fewer than 100 blocks |
| Non-overlapping Template | Template length m | 9 | m = 9 or 10 gives meaningful results; each template gets its own P-value (up to 148 for m = 9) |
| Overlapping Template | Template length m | 9 | Runs of m ones; block size and other internals are fixed in the code |
| Approximate Entropy | Block length m | 10 | Keep m at or below $\log_2(n)$; results drift from expected values beyond about m = 14 |
| Serial | Block length m | 16 | Valid for $m \leq \log_2(n) - 2$, which is 17 for n = 1,000,000 |
| Linear Complexity | Block length M | 500 | Use $500 \leq M \leq 5000$ |

NIST's overall rule of thumb is to pick block sizes no larger than $\log_2(n)$. Note that NIST's sample runs use M = 128 for Block Frequency even though its written guidance asks for fewer than 100 blocks, which for 1,000,000 bits would mean a much larger M. Treat the defaults as the way to reproduce NIST's published numbers, and the written guidance as the safer choice for a careful analysis.

The other tests need no choices. Longest Run of Ones picks its block size from n (8 bits from n = 128, 128 bits from n = 6,272, 104 bits from n = 750,000). Rank always uses 32×32 matrices. Maurer's Universal test chooses its block length L from n.

### Minimum sequence lengths

A test run on too few bits gives unreliable P-values, because the tests rely on approximations that only hold for long sequences. With n = 1,000,000 you will satisfy nearly all of them.

## Step 5: Reading the results

A generator passes when two checks hold for every test: enough sequences pass, and the P-values are spread evenly between 0 and 1. The suite calculates both for you in one summary file.

```
cat experiments/AlgorithmTesting/finalAnalysisReport.txt
```

Each row is one test. Here is a row in the layout NIST shows in its guide:

```
 C1  C2  C3  C4  C5  C6  C7  C8  C9  C10  P-VALUE    PROPORTION  STATISTICAL TEST
  6  12   9  12   8   7   8  12  15   11  0.616305   99/100      Frequency
```

| Column | Meaning |
|---|---|

| Column | Meaning |
|---|---|
| C1 to C10 | A histogram: how many sequences had a P-value in 0–0.1, 0.1–0.2, and so on up to 0.9–1.0 |
| P-VALUE | A P-value for the histogram itself, testing whether the 10 bins are evenly filled |
| PROPORTION | How many sequences passed that test (newer releases print a fraction like 99/100; older ones print 0.9900) |
| STATISTICAL TEST | The test name; a test that gives two P-values per sequence, such as Cusum or Serial, gets two rows |

**Check 1: the proportion of sequences passing.** At $\alpha = 0.01$ you expect about 99% of sequences to pass. Typical values at $\alpha = 0.01$:

| Sequences (m) | Minimum acceptable pass rate |
|---|---|
| 55 | about 0.950 (0.94975) |
| 100 | 0.960 (96.015%, as printed in NIST's example) |
| 1,000 | 0.981 (98.056%) |

**Sources**

- NIST SP 800-22 Rev. 1a: A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications
- NIST SP 800-22: Documentation and Software (CSRC)